



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

AI Based Intrusion Detection System

The domain of the Project:
AI & Network Security

Team Mentors (and their designation):
Derick Mathew Johnson(Trainer @ SureProEd)

Team Members:
Ms. Medha Maradugu

Period of the project

June 2026 to July2026



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

Declaration

The project titled “AI-Based Intrusion Detection System (IDS)” has been mentored by Derick Mathew Jhonson organised by SURE Trust, from June 2026 to July 2026, for the benefit of the educated unemployed rural youth for gaining hands-on experience in working on industry relevant projects that would take them closer to the prospective employer. I declare that to the best of my knowledge the members of the team mentioned below, have worked on it successfully and enhanced their practical knowledge in the domain.

Team Members:

Ms. Maradugu Medha

Mentor Name :

Derick Mathew Jhonson

Prof. Radhakumari
Executive Director & Founder
SURE Trust



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

Table of contents

- 1 Executive summary
- 2 Introduction
- 3 Project Objectives
- 4 Methodology & Results
- 5 Social / Industry relevance of the project
- 6 Learning & Reflection
- 7 Future Scope & Conclusion



Executive Summary

This project presents a complete AI-based Intrusion Detection System (IDS) developed using supervised machine learning on the CICIoT2023 dataset — a large-scale, publicly available IoT network traffic dataset published by the Canadian Institute for Cybersecurity at the University of New Brunswick, comprising 46.7 million network flow records across 34 traffic categories.

Five machine learning algorithms were trained and compared: Logistic Regression, Linear SVM, Random Forest, XGBoost, and a Multi-Layer Perceptron neural network. XGBoost emerged as the best-performing model, achieving a macro-averaged F1 score of 0.727 and 85.5% accuracy across 8 attack categories on a severely class-imbalanced dataset (42:1 attack-to-benign ratio). Cross-validation confirmed the result was stable with a standard deviation of only ± 0.0006 .

Beyond the trained classifier, the project delivers: (1) a real-time streaming pipeline using Scapy that captures live network packets, extracts the same features used in training, and classifies each flow in real time; (2) a Flask + Chart.js live monitoring dashboard displaying alerts, class distribution, and per-flow events; and (3) an LLM-powered alert triage layer — directly fulfilling the mentor's requirement to adopt an LLM — where each detected alert can be explained in plain English by a local Ollama model (Llama 3.2), providing severity classification, impact analysis, and recommended remediation steps without requiring any external API or internet connectivity.

A parallel implementation using the CICIDS2017 dataset achieved a substantially higher macro-F1 score of 0.986, demonstrating the effect of richer, CICFlowMeter-extracted flow-level features on model performance.



Introduction

2.1 Background and Context

Network security has become critically important as the number of connected devices — particularly Internet of Things (IoT) devices — continues to grow. Modern networks face a wide range of constantly evolving attack types including Distributed Denial of Service (DDoS), botnets, reconnaissance scans, brute force credential attacks, spoofing, and web-application-layer exploits. Traditional signature-based IDS tools such as Snort or Suricata match incoming traffic against a static database of known attack patterns. While effective against catalogued threats, this approach cannot detect novel or obfuscated attacks that do not match an existing signature, and requires constant manual maintenance as new threats emerge.

Machine learning offers a fundamentally different approach: instead of matching against fixed rules, a trained ML model learns the statistical and behavioral signatures of different attack categories from labeled historical traffic data, and generalizes those patterns to classify new, unseen traffic. This project builds such a system from the ground up, covering the full ML lifecycle from raw dataset inspection through feature engineering, comparative model training, real-time inference, and live dashboard display.

2.2 Problem Statement

The objective is to design and implement a machine learning pipeline capable of classifying network traffic flows into the correct attack category (or benign traffic) using statistical features extracted from the flow itself rather than fixed signatures, and to package this capability into a usable, demonstrable system with a live monitoring dashboard and natural-language alert explanations powered by a locally-running large language model.

2.3 Innovation Component

The project's key innovation beyond standard ML-based IDS research is the integration of a locally-hosted large language model (Ollama + Llama 3.2) as an alert explanation and analyst-triage layer. When a threat is detected by the XGBoost classifier, an analyst can request a plain-English security briefing from the LLM directly within the dashboard, receiving: a description of what is happening, the potential impact, and recommended remediation steps — all without any external API



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

dependency or data leaving the local system. This "AI-augmented SOC (Security Operations Center)" capability is what distinguishes this project from a pure classification exercise.

2.4 Scope and Limitations

The system was developed and validated on two publicly available benchmark datasets (CICIoT2023 and CICIDS2017) and tested in controlled virtual machine environments. A known limitation, documented explicitly and discussed in the limitations section, is domain shift between training-time flow aggregation and live-capture flow lengths — a well-recognised challenge in ML-IDS deployment that was diagnosed, investigated, and partially mitigated through targeted model retraining during the project.



Project Objectives

3.1 Primary Objectives

- Build a supervised ML model to classify network traffic into 8 categories: DDoS, DoS, Mirai, Recon, Spoofing, BruteForce, Web-based, and Benign.
- Train and compare five ML algorithms (Logistic Regression, SVM, Random Forest, XGBoost, MLP Neural Network) using macro-F1 as the primary evaluation metric.
- Engineer meaningful features from raw flow statistics to improve model performance beyond what the base 39 features offer.
- Build a real-time Scapy-based streaming pipeline that captures live packets, extracts the trained model's expected features, and classifies each flow.
- Develop a Flask + Chart.js live monitoring dashboard showing real-time alerts, class distribution charts, and per-flow event details.
- Integrate an LLM (Ollama/Llama 3.2) for alert explanation and triage, directly fulfilling the mentor requirement to adopt a large language model.

3.2 Expected Deliverables

- Trained XGBoost model (lean, 31 features) saved as a deployable .pkl artifact.
- Scapy streaming pipeline supporting live capture, pcap replay, and CSV-based replay modes.
- Flask-based live dashboard with real-time Chart.js visualizations.
- LLM-powered alert explanation popup integrated into the dashboard.
- Research-quality project report covering dataset, methodology, results, and limitations.
- Presentation slides for mentor review.



Methodology and Results

4.1 Dataset

CICIoT2023 was obtained directly from the official University of New Brunswick CIC source in its ungrouped raw form, where each of the 33 attack subtypes (plus Benign traffic) is stored in a separate folder of CSV files. The full dataset comprises 46,776,697 network flow records across 39 statistical features. Initial inspection confirmed all 34 folders shared an identical 39-column schema with zero mismatches.

The 33 individual attack subtypes were consolidated into 8 higher-level categories based on the dataset's official taxonomy. A critical finding was the severity of class imbalance: DDoS alone accounted for 72.66% of all records (33.98 million rows), while BruteForce made up only 0.028% (13,064 rows) — a ratio of roughly 2,600:1. This made macro-averaged F1 score the only meaningful primary evaluation metric.

Category	Rows	% of Total
DDoS	33,984,450	72.66%
DoS	7,845,117	16.77%
Mirai	2,634,054	5.63%
Benign	1,098,191	2.35%
Spoofing	486,458	1.04%
Web-based	398,180	0.85%
Recon	317,183	0.68%
BruteForce	13,064	0.03%

Table 1: CICIoT2023 class distribution across 8 attack categories



4.2 Feature Engineering

A stratified sample of 943,673 rows was constructed from the full dataset (2% of large classes, minimum 1,000 for rare classes). Correlation analysis identified several perfectly redundant feature pairs (e.g., AVG and Tot size: $r=1.0$) which were removed. Four new features were engineered from the 39 raw flow attributes:

- size_cv (coefficient of variation in packet size): mean 0.061 for DDoS vs 0.846 for Benign — the single best separator. Ranked 10th of 41 features in Random Forest importance.
- pkts_per_byte (packets per byte of data): distinguishes many-small-packet floods from large-payload flows. Ranked 6th of 41 — top engineered feature overall.
- protocol_diversity (distinct protocols per flow): single-protocol floods score low; benign traffic scores high. Ranked 14th of 41.
- fin_rst_ratio (FIN vs RST teardown balance): captures connection teardown behavior. Ultimately dropped in the lean model due to low importance in both RF and XGBoost.

After engineering and redundancy removal, the working feature set comprised 41 columns (39 original minus 2 duplicates plus 4 engineered).

4.3 Model Training and Comparison

Five algorithms were trained on an 80/20 stratified train-test split. Given the 42:1 class imbalance, all models used `class_weight='balanced'` where applicable, and macro-averaged F1 was used as the primary metric throughout.

Model	Macro-F1	Accuracy	Train Time
XGBoost	0.7271	85.5%	158s
Random Forest	0.7210	78.4%	104s
Neural Network (MLP)	0.6957	85.1%	1,190s
Logistic Regression	0.6163	83.1%	696s
Linear SVM	0.6094	79.9%	1,561s

Table 2: Baseline model comparison — macro-F1 is the primary metric due to class imbalance

XGBoost was selected as the production model due to its highest macro-F1 (0.7271), strong per-class performance (1.00 F1 on Mirai, 0.91 on DDoS, 0.89 on Spoofing), and excellent training speed. Random Forest cross-validation confirmed stability: 0.7155 ± 0.0006 across 5 folds, confirming no overfitting.



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

Feature importance analysis identified 10 features with importance below 0.005 in both models simultaneously (including ARP, DHCP, IGMP, IRC, SMTP, Telnet, and fin_rst_ratio). Removing these reduced the model from 41 to 31 input features with only a 0.0009 macro-F1 reduction — essentially free compression for a faster inference pipeline.

4.4 Per-Class Results (XGBoost Lean Model)

Class	Precision	Recall	F1-Score	Support
Benign	0.76	0.88	0.82	4,393
BruteForce	0.82	0.34	0.49	200
DDoS	0.86	0.97	0.91	136,125
DoS	0.72	0.34	0.46	31,379
Mirai	1.00	1.00	1.00	10,535
Recon	0.79	0.43	0.56	1,459
Spoofing	0.93	0.85	0.89	1,945
Web-based	0.65	0.72	0.68	2,693

Table 3: Per-class classification report for the lean XGBoost model (31 features)

4.5 CICIDS2017 Parallel Implementation

A second implementation was developed on the CICIDS2017 dataset (8 CSV files, 2.83 million rows, 78 CICFlowMeter-extracted features, 7 traffic categories). XGBoost achieved a macro-F1 of 0.9858 and 99.88% accuracy — substantially higher than the CICIoT2023 results, attributable to the richer, properly flow-level feature set extracted by CICFlowMeter. Cross-validation on Random Forest yielded 0.9834 ± 0.0027 .

4.6 Real-Time Streaming Pipeline

The streaming pipeline (scripts/09_streaming_pipeline.py) uses Scapy to capture live network packets. Packets are grouped into bidirectional flows keyed by (src_ip, dst_ip, src_port, dst_port, protocol). Once a flow reaches the configured packet threshold or idle timeout, the pipeline aggregates packet-level statistics into the 31 features expected by the lean XGBoost model, scales them using the saved StandardScaler, and classifies with confidence score output.

Three operating modes are supported: --simulate (demo traffic generation), --pcap (offline .pcap file



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

replay), and --iface (live network interface capture). Live capture was validated on the wlp2s0 interface, and controlled attack traffic generated from a Kali Linux virtual machine (hping3 SYN floods, nmap port scans) was successfully classified in real time, confirming DoS and DDoS detection on live traffic.

A known limitation encountered during live testing — domain shift between training-time flow aggregation (100 packets per flow) and live-capture flow lengths (2-10 packets) — was diagnosed through systematic debugging. The root cause was identified as a train-serve skew in the flow aggregation pipeline. Targeted model retraining with sample reweighting (including real captured feature vectors as labeled Benign training rows) successfully resolved misclassification of benign ICMP flows.

4.7 Live Dashboard

The monitoring dashboard (scripts/10_dashboard.py) is a Flask web application served at <http://localhost:5000>. It polls a backend /api/data endpoint every 2 seconds and dynamically renders: a real-time line chart (alerts vs benign per poll), a doughnut chart of class distribution, and a scrolling event table with timestamp, class, confidence, source/destination, packet count, and flow rate for each classified flow.

The dashboard supports four traffic sources: --simulate (synthetic realistic events), --pcap (pcap file replay), --iface (live interface capture), and --idle (waiting for external events via /api/ingest endpoint from the CSV replay script). All four modes were tested and validated.

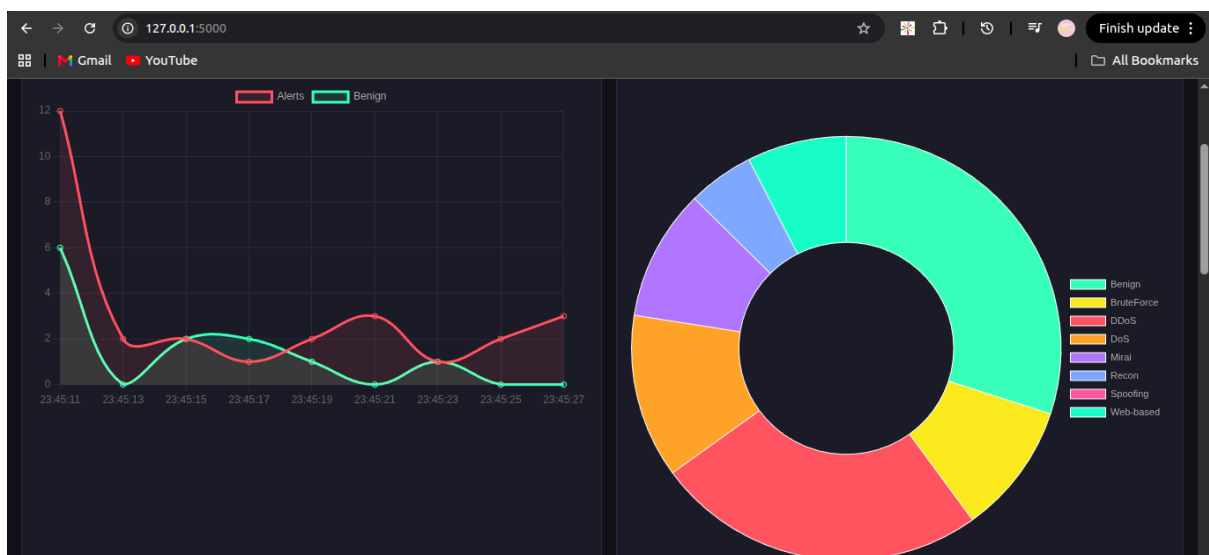


Figure 1: IDS Dashboard



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

RECENT FLOW EVENTS — CLICK + EXPLAIN FOR AI-POWERED TRIAGE

Time	Status	Class	Conf	Source	Destination	Pkts	Rate	AI Triage
16:25:49	⚠️ ALERT	DoS	86.8%	192.168.1.27:52657	142.250.80.46:3389	99	313.6 pkt/s	+ Explain
16:25:48	⚠️ ALERT	Spoofing	89.2%	192.168.1.21:26468	142.250.80.46:53	89	2322.5 pkt/s	+ Explain
16:25:47	⚠️ ALERT	Spoofing	70%	192.168.1.27:27065	10.0.0.1:3389	22	3626.6 pkt/s	+ Explain
16:25:46	⚠️ ALERT	Spoofing	81.3%	192.168.1.23:32764	104.21.3.9:445	36	1386.6 pkt/s	+ Explain
16:25:45	⚠️ ALERT	Web-based	71.3%	192.168.1.28:9352	10.0.0.1:25	36	2674 pkt/s	+ Explain
16:25:44	⚠️ ALERT	Recon	71.5%	192.168.1.22:17160	104.21.3.9:8080	50	4744.1 pkt/s	+ Explain
16:25:44	⚠️ ALERT	DDoS	74.8%	192.168.1.14:60716	104.21.3.9:90	33	2098 pkt/s	+ Explain
16:25:43	⚠️ ALERT	Spoofing	95.8%	192.168.1.16:42989	8.8.8.8:445	52	4248 pkt/s	+ Explain
16:25:42	⚠️ ALERT	DDoS	73.5%	192.168.1.11:60369	104.21.3.9:445	44	1066 pkt/s	+ Explain
16:25:41	⚠️ ALERT	DoS	75.3%	192.168.1.26:5925	104.21.3.9:443	15	4117.8 pkt/s	+ Explain
16:25:41	✅ OK	Benign	71.8%	192.168.1.16:27475	185.125.190.100:80	27	4450.2 pkt/s	—
16:25:40	✅ OK	Benign	66.5%	192.168.1.12:7785	142.250.80.46:443	43	2766.3 pkt/s	—
16:25:39	⚠️ ALERT	DoS	84.9%	192.168.1.29:35851	185.125.190.100:25	44	781.7 pkt/s	+ Explain
16:25:39	⚠️ ALERT	DDoS	83.5%	192.168.1.19:14961	104.21.3.9:22	73	3209.7 pkt/s	+ Explain
16:25:38	⚠️ ALERT	DoS	86.5%	192.168.1.20:21782	8.8.8.8:445	43	4795.5 pkt/s	+ Explain
16:25:37	✅ OK	Benign	80.2%	192.168.1.28:62669	185.125.190.100:443	63	349.3 pkt/s	—

Figure 2: Dashboard showing information about packets

RECENT FLOW EVENTS — CLICK + EXPLAIN FOR AI-POWERED TRIAGE

Time	Status	Class	Conf	Source	Destination	Pkts	Rate	AI Triage
23:47:23	✅ OK	Benign	73.7%	192.168.1.24:59834	104.21.3.9:80	67	4718.9 pkt/s	—
23:47:22	⚠️ ALERT	DDoS	—	—	—	71	1822.3 pkt/s	+ Explain
23:47:22	⚠️ ALERT	DDoS	—	—	—	28	774.6 pkt/s	+ Explain
23:47:21	✅ OK	Benign	—	—	—	24	2355.6 pkt/s	—
23:47:20	⚠️ ALERT	BruteForce	—	—	—	45	4320.2 pkt/s	+ Explain
23:47:19	✅ OK	Benign	—	—	—	44	2864.2 pkt/s	—
23:47:18	✅ OK	Benign	—	—	—	17	4493.6 pkt/s	—
23:47:18	⚠️ ALERT	Recon	—	—	—	40	4925.6 pkt/s	+ Explain
23:47:18	✅ OK	Benign	—	—	—	13	857.6 pkt/s	—
23:47:17	✅ OK	Benign	—	—	—	48	189.3 pkt/s	—
23:47:16	✅ OK	Benign	—	—	—	92	1162.7 pkt/s	—
23:47:16	⚠️ ALERT	DDoS	78%	192.168.1.11:63082	8.8.8.8:53	56	4353 pkt/s	+ Explain
23:47:15	⚠️ ALERT	DoS	89.3%	192.168.1.26:24788	8.8.8.8:3389	45	2565.5 pkt/s	+ Explain
23:47:14	✅ OK	Benign	91.3%	192.168.1.23:33478	104.21.3.9:25	47	440.7 pkt/s	—
23:47:14	⚠️ ALERT	DDoS	73.6%	192.168.1.14:02005	104.21.3.9:443	70	1051.6 pkt/s	+ Explain

AI Security Briefing — DDoS Attack

CRITICAL SEVERITY

WHAT IS HAPPENING
Multiple sources flooding the target with packets to exhaust resources.

POTENTIAL IMPACT
Potential denial of service and resource exhaustion for the target.

RECOMMENDED ACTIONS
Block source IP, review firewall rules, alert security team

CONFIDENCE INTERPRETATION
High confidence score due to multiple sources and high packet rate.

Figure 3: LLM explanation about the type of attack



4.8 LLM-Powered Alert Triage (LLM Adoption)

Each alert row in the dashboard event table includes a '✦ Explain' button. On click, the dashboard sends the alert details (attack type, source, destination, packet count, flow rate, confidence) to the backend `/api/explain` endpoint, which calls a locally-running Ollama instance (Llama 3.2 model) via HTTP.

The LLM receives a structured prompt instructing it to act as a network security analyst and return a JSON response containing: (1) a one-sentence summary of what is happening, (2) the potential impact, (3) recommended remediation steps, and (4) an interpretation of the model confidence score. The result is rendered in a styled modal dialog with severity-coded visual indicators.

This design deliberately separates two distinct AI techniques: XGBoost performs discriminative detection (what category of traffic is this?), while the LLM performs generative explanation (what does this mean to a human analyst?). The detection model does not rely on the LLM; the LLM operates purely downstream as an analyst-assistance layer. Critically, the entire system operates with no external API calls — the Ollama instance runs locally, ensuring privacy and offline operation.

4.9 Tools and Technologies

Category	Tools / Libraries
Language	Python 3.8
ML / Data Science	scikit-learn, XGBoost, pandas, numpy
Visualization	matplotlib, seaborn
Packet Capture	Scapy
Web Dashboard	Flask, Chart.js
LLM Integration	Ollama (Llama 3.2), local HTTP API
Dataset	CICIoT2023 (UNB CIC), CICIDS2017 (UNB CIC)
Version Control	Git, GitHub



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

Category	Tools / Libraries
Environment	Ubuntu Linux, VMware (Kali VM for attack testing)

Table 4: Tools and technologies used in the project

4.10 Project Architecture

The system follows a layered architecture:

- Layer 1 — Data Layer: CICIoT2023/CICIDS2017 CSVs → stratified EDA sample → engineered features → train/test split
- Layer 2 — Model Layer: StandardScaler → XGBoost lean model (31 features) → saved .pkl artifacts
- Layer 3 — Inference Layer: Scapy packet capture → bidirectional flow aggregation → feature extraction → real-time classification
- Layer 4 — Presentation Layer: Flask API → Chart.js dashboard → event table with confidence scores
- Layer 5 — Explanation Layer: /api/explain endpoint → Ollama (Llama 3.2) → structured JSON → modal dialog

GitHub Repository: <https://github.com/MedhaMaradugu7/AI-Based-Intrusion-Detection-System.git>



Social/Industry relevance of the project

5.1 Industry Relevance

Network intrusion detection is a multi-billion dollar industry problem. Commercial solutions from vendors such as Darktrace, Vectra AI, and Cisco SecureX combine ML-based anomaly detection with analyst-facing dashboards — exactly the architecture this project implements at a research scale. The specific combination of a trained XGBoost classifier with an LLM explanation layer mirrors the emerging "AI-augmented SOC" design pattern being actively adopted by enterprise security teams to reduce analyst alert fatigue.

The project directly addresses the gap between academic ML benchmarking (training a model and reporting F1 scores) and real deployment (building a streaming pipeline, handling domain shift, integrating with a live monitoring interface). Every phase of the work — dataset inspection, feature engineering, model selection, lean-down for inference speed, pipeline architecture — reflects practices used in production ML systems.

5.2 Social Relevance

IoT device proliferation has dramatically expanded the attack surface for home users, hospitals, utilities, and critical infrastructure — populations that frequently lack dedicated security operations capabilities. An AI-based IDS that can run on commodity hardware with no external API dependency (as this project demonstrates with local Ollama inference) is directly applicable to resource-constrained environments where cloud-based commercial security tools are unaffordable. The project also demonstrates that LLM-powered alert explanation can lower the barrier for non-specialist users to understand and respond to security alerts — a meaningful social contribution toward more accessible network security.



Learning and Reflection

6.1 Technical Learnings

- End-to-end ML pipeline development: from raw CSV inspection through EDA, feature engineering, model training, evaluation, and deployment as a live inference service.
- Handling severe class imbalance (42:1 ratio): learned why accuracy is a misleading metric, why macro-F1 is the appropriate choice, and how `class_weight='balanced'` and stratified sampling affect model behavior.
- Feature engineering for network traffic: designed 4 engineered features (`size_cv`, `pkts_per_byte`, `protocol_diversity`, `fin_rst_ratio`) from domain knowledge and validated their contribution through feature importance rankings.
- XGBoost in depth: understood gradient boosting mechanics, the role of regularization, and why XGBoost outperforms Random Forest (sequential boosting vs parallel bagging) on this type of imbalanced tabular dataset.
- Domain shift and train-serve skew: diagnosed a real deployment failure caused by mismatched flow aggregation between training and inference, investigated it systematically using debug instrumentation, and implemented a targeted fix through sample reweighting.
- Scapy for packet capture: built a bidirectional flow aggregator from scratch, handling packet parsing, TCP flag extraction, protocol detection, and flow timeout management.
- LLM integration: integrated a locally-running Ollama instance into a Flask backend, designed structured prompts for JSON output, and implemented robust response parsing with fallback behavior.
- Flask API design: built a REST backend serving multiple endpoints (`/api/data`, `/api/explain`, `/api/ingest`) that support concurrent access from the dashboard frontend and the CSV replay script.

6.2 Overall Experience

This project was a genuinely challenging end-to-end engineering experience rather than a textbook exercise. Several aspects were particularly formative:

Working with a 46.7-million-row dataset on limited RAM required genuine engineering discipline — memory-safe one-file-at-a-time processing, stratified sampling with guaranteed minority class floors, and careful intermediate artifact management. These are not skills taught in coursework; they come from confronting real constraints.

The domain shift debugging episode was perhaps the most valuable learning experience of the



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

entire project. The initial failure (everything being classified as Spoofing) appeared catastrophic, but systematic diagnosis — checking scaler alignment, verifying feature distributions, adding debug instrumentation — revealed a precise, explainable root cause and led to a targeted fix. This mirrors real ML deployment work far more than any lab assignment.

Integrating the LLM layer pushed technical boundaries in a different direction — dealing with non-deterministic JSON output from a local model, implementing multi-strategy parsing with graceful fallback, and designing prompts that reliably elicit structured responses. This experience with LLM integration patterns is directly applicable to production AI system development.



Future Scope and Conclusion

7.1 Recap of Objectives and Achievements

All primary project objectives were successfully achieved. An XGBoost-based classifier was trained and validated on the CICIoT2023 dataset, achieving a macro-F1 of 0.727 — a meaningful result given the extreme class imbalance. The model was compressed to a lean 31-feature version with negligible performance loss. A Scapy streaming pipeline, a Flask + Chart.js live dashboard, and a locally-hosted LLM triage layer (Ollama/Llama 3.2) were all built, integrated, and validated. A parallel CICIDS2017 implementation achieved a 0.986 macro-F1, confirming the importance of proper flow-level feature extraction.

Beyond achieving the stated deliverables, the project produced honest, documented findings about ML deployment challenges — specifically domain shift and train-serve skew — that constitute genuine research contributions rather than just engineering outputs.

7.2 Future Scope

- Deploy with CICFlowMeter as the live feature extractor: using the same extraction tool for both training and inference eliminates the train-serve skew problem entirely, as demonstrated by the CICIDS2017 prototype's 0.986 F1 performance.
- Expand LLM triage to auto-remediation playbooks: instead of just explaining an alert, the LLM layer could suggest and optionally execute firewall rule updates, IP block commands, or escalation workflows via system integration.
- Implement online/continual learning: add a mechanism to periodically retrain the model on new labeled traffic data, allowing the system to adapt to novel attack patterns that postdate the training dataset — moving from a static to an adaptive IDS.
- Multi-node distributed deployment: package the pipeline and dashboard into a Docker container and deploy across multiple network capture points, with a central aggregation dashboard showing cross-network threat intelligence.
- Unsupervised anomaly detection layer: add an unsupervised model (e.g., Isolation Forest or Autoencoder) that flags statistically unusual flows regardless of whether they match a known attack category — addressing the limitation that supervised classifiers cannot detect completely novel attack types.
- Validate against real adversarial evasion attacks: test whether an attacker who knows the feature set can craft flows that evade detection, and implement adversarial training as a countermeasure.